计算机学院实验报告

实验题目： Mipmap 实现		学号：202200130077
日期：2024.5.14	班级：智能	姓名：薛润泽
Email：		

实验目的：

实验内容：

1， 任选一张 1024*1024 的图像，非实验 6 中提供的纹理图片

2， 生成 Mipmap 序列图像

3， 实现 Mipmap 序列图像的 Trilinear 插值

4， 应用于实验 6 提供的任意模型的纹理贴图

实验环境介绍：

ubuntu

解决问题的主要思路：

本实验的主要思路如下：

1. 对于生成 Mipmap 序列图像，考虑递归实现，当前所处理的图像是 Width 和 Height 上一次结束处理图像的 1/2，当前图像的一个像素相当于上一次结束处理图像的 4 个像素，所以遍历当前图像每个像素，赋值为上次图像对应 4 个像素的平均值。最后从 1024×1024 生成到 1×1，最后会生成共计 10 张图片。
2. 对于 Trilinear 插值，首先计算出当前屏幕像素点对应到原始纹理图片中，对应正方形块，块长为 L。通过求 log（L），找到两个 Level，这两个 Level 就是我们需要找像素点的两个层级。在这两个层级中使用双线性插值求出一个模型对应纹理 color 值，再将这两个 color 进行一次插值，就完成了 Trilinear 插值。

3、最后，通过 Trilinear 插值的方式应用与 texture 纹理点取色中。

实验步骤与实验结果：

1. 生成 Mipmap 图像，递归实现生成，对上一张图片的每相邻四个像素点取平均值，赋给该像素点。

2、Trilinear 插值，生成 Mipmap 序列后，将序列图像信息存入数组 cv::Mat image_DATA[12]中。记录两个全局变量 last_x, last_y，用来记录上次遍历到的像素点，方便与此次遍历到的像素点位置做差，从而计算 L，来得出此次使用 Mipmap 序列中的哪一层。通常计算出 L 后，求 log(L)往往不是整数，所以对层求双线性插值后，再通过 L 与 floor(L)的差值 t，对两次双线性插值求出的两个 color 再进行一次 t*color1 + （1 - t）*color2 的插值，从而完成 Trilinear 插值。
下面是代码实现，float last_x, last_y 是定义在 main.cpp 中的两个全局变量，用来记录上次遍历的像素位置，便于计算 L。cv:: Mat image_DATA[12]数组用来记录生成的 Mipmap 图像信息。为了便于使用 Mipmap 图像信息，我重新写了一个 getColorBiliear_2(float u, float v, const cv::Mat &image)函数，用于 Trilinear 中的双线性插值：

```
// Texture.hpp 中的 Texture类
class Texture{
private:
    cv::Mat image_data;
    cv::Mat image_DATA[12];
// Public 中的 Texture()构造函数中 写了如何生成 Mipmap序列并记录序列信息

public:

    Texture(const std::string& name)
    {
        image_data = cv::imread(name);
        cv::cvtColor(image_data, image_data,
cv::COLOR_RGB2BGR);
        image_DATA[0] = image_data;
        width = image_data.cols;
        height = image_data.rows;
        auto L = width / 2;
        auto count = 1;
        while(L > 0){
            cv::Mat img(L, L, CV_8UC3, cv::Scalar(0,0,0));
            std::string s_ready = "image", s_last = "image",  s1
= ".png";
            s_ready += '0' + count;  s_last += '0' + count - 1;
            s_ready+= s1; s_last += s1;
```

```
            cv::Mat img2 = cv::imread(s_last, cv::IMREAD_COLOR);
            for(auto i = 0; i < img.rows; i++){
                for(auto j = 0; j < img.cols; j++){
                    auto ri = 2 * (i + 1), rj = 2*(j + 1);
                    auto color = 1.0f *img2.at<cv::Vec3b>(ri -
1 , rj - 1) / 4 + 1.0f * img2.at<cv::Vec3b>(ri - 1 , rj - 1 -
1) / 4 +  img2.at<cv::Vec3b>(ri - 1 - 1 , rj - 1) / 4
+img2.at<cv::Vec3b>(ri - 1 - 1 , rj - 1 -1) / 4;
                    img.at<cv::Vec3b>(i, j) = color;
                }
            }
            cv::imwrite(s_ready, img);
            image_DATA[count] = cv::imread(s_ready);
            cv::cvtColor(image_DATA[count], image_DATA[count],
cv::COLOR_RGB2BGR);
            count++;
            L  = L / 2;
        }
    }

    int width, height;
    Eigen::Vector3f getColor(float u, float v)
    {
        u = std:: max(0.0f, u); u = std::min(1.0f, u);
        v = std:: max(0.0f, v); v = std::min(1.0f, v);
        auto u_img = u * width;
        auto v_img = (1 - v) * height;
        auto color = image_data.at<cv::Vec3b>(v_img, u_img);
        return Eigen::Vector3f(color[0], color[1], color[2]);
    }

    Eigen::Vector3f getColorBilinear(float u, float v){
        u = std:: max(0.0f, u); u = std::min(1.0f, u);
        v = std:: max(0.0f, v); v = std::min(1.0f, v);
        auto u_img = u * width;
        auto v_img = (1 - v) * height;
        auto u_img1 = (int)(u_img);
        auto u_img2 = u_img1 + 1;
        auto v_img1 = (int)(v_img);
        auto v_img2 = v_img1 + 1;
        float k1 = v_img - v_img1;
        float k2 = u_img - u_img1;
        auto color1 = image_data.at<cv::Vec3b>(v_img1, u_img1);
        auto color2 = image_data.at<cv::Vec3b>(v_img2, u_img1);
```

```
        auto color3 = image_data.at<cv::Vec3b>(v_img1, u_img2);
        auto color4 = image_data.at<cv::Vec3b>(v_img2, u_img2);

        auto color5 = k1 * color1 + (1.0f - k1) * color2;
        auto color6 = k1 * color3 + (1.0f - k1) * color4;
        auto color = k2 * color5 + (1.0f - k2) * color6;
        return Eigen::Vector3f(color[0], color[1], color[2]);
    }
```

// 为了Trilinear插值定制的 getColorBilinear_2 函数，用于不同纹理图片的双线性插值

```
    Eigen::Vector3f getColorBilinear_2(float u, float v, const
cv::Mat &image){
        u = std:: max(0.0f, u); u = std::min(1.0f, u);
        v = std:: max(0.0f, v); v = std::min(1.0f, v);
        auto u_img = u * image.cols;
        auto v_img = (1 - v) * image.rows;
        auto u_img1 = (int)(u_img);
        auto u_img2 = u_img1 + 1;
        auto v_img1 = (int)(v_img);
        auto v_img2 = v_img1 + 1;
        float k1 = v_img - v_img1;
        float k2 = u_img - u_img1;
        auto color1 = image.at<cv::Vec3b>(v_img1, u_img1);
        auto color2 = image.at<cv::Vec3b>(v_img2, u_img1);

        auto color3 = image.at<cv::Vec3b>(v_img1, u_img2);
        auto color4 = image.at<cv::Vec3b>(v_img2, u_img2);

        auto color5 = k1 * color1 + (1.0f - k1) * color2;
        auto color6 = k1 * color3 + (1.0f - k1) * color4;
        auto color = k2 * color5 + (1.0f - k2) * color6;
        return Eigen::Vector3f(color[0], color[1], color[2]);
    }
    Eigen::Vector3f getColor_Trilinear(float u, float v, float
last_x, float last_y){
        u = std:: max(0.0f, u); u = std::min(1.0f, u);
        v = std:: max(0.0f, v); v = std::min(1.0f, v);
        last_x = std:: max(0.0f, last_x); last_x =
std::min(1.0f, last_x);
        last_y = std:: max(0.0f, last_y); last_y =
std::min(1.0f, last_y);
        if((std::abs(last_x + 1) < 0.0001)  && std::abs(last_y
+ 1) < 0.0001){
```

```
            return getColorBilinear(u, v);
        }
        else{
            auto u_img = u * width;
            auto v_img = (1 - v) * height;
            auto x_img = last_x * width;
            auto y_img = (1 - last_y) * height;

            auto L = 1.0 * std::sqrt((x_img - u_img) * (x_img -
u_img) + (v_img - y_img) * (v_img - y_img));
            auto level = std::log2(L);
            auto FLOOR = std::floor(level);
            auto CEIL = std::ceil(level);
            auto t = level - FLOOR;
            if(FLOOR < 0 || CEIL >10 || t < 0 || t > 1) return
getColorBilinear(u, v);
            // printf("right in \n");
            auto color1 = getColorBilinear_2(u, v,
image_DATA[(int)FLOOR]);
            auto color2 = getColorBilinear_2(u, v,
image_DATA[(int)CEIL]);
            auto color3 = t * color1 + (1 - t)*color2;
            return
Eigen::Vector3f(color3[0],color3[1],color3[2]);
        }
    }
};
```

下面是在 main.cpp texture_fragment_shader 函数中，调用 Mipmap 进行纹理取样的两行代码

```
        return_color =
payload.texture->getColor_Trilinear(payload.tex_coords.x(),
payload.tex_coords.y(), last_x, last_y);
        last_x = payload.tex_coords.x(); last_y =
payload.tex_coords.y();
```

实验效果：

Mipmap 序列图片如下：

Image0

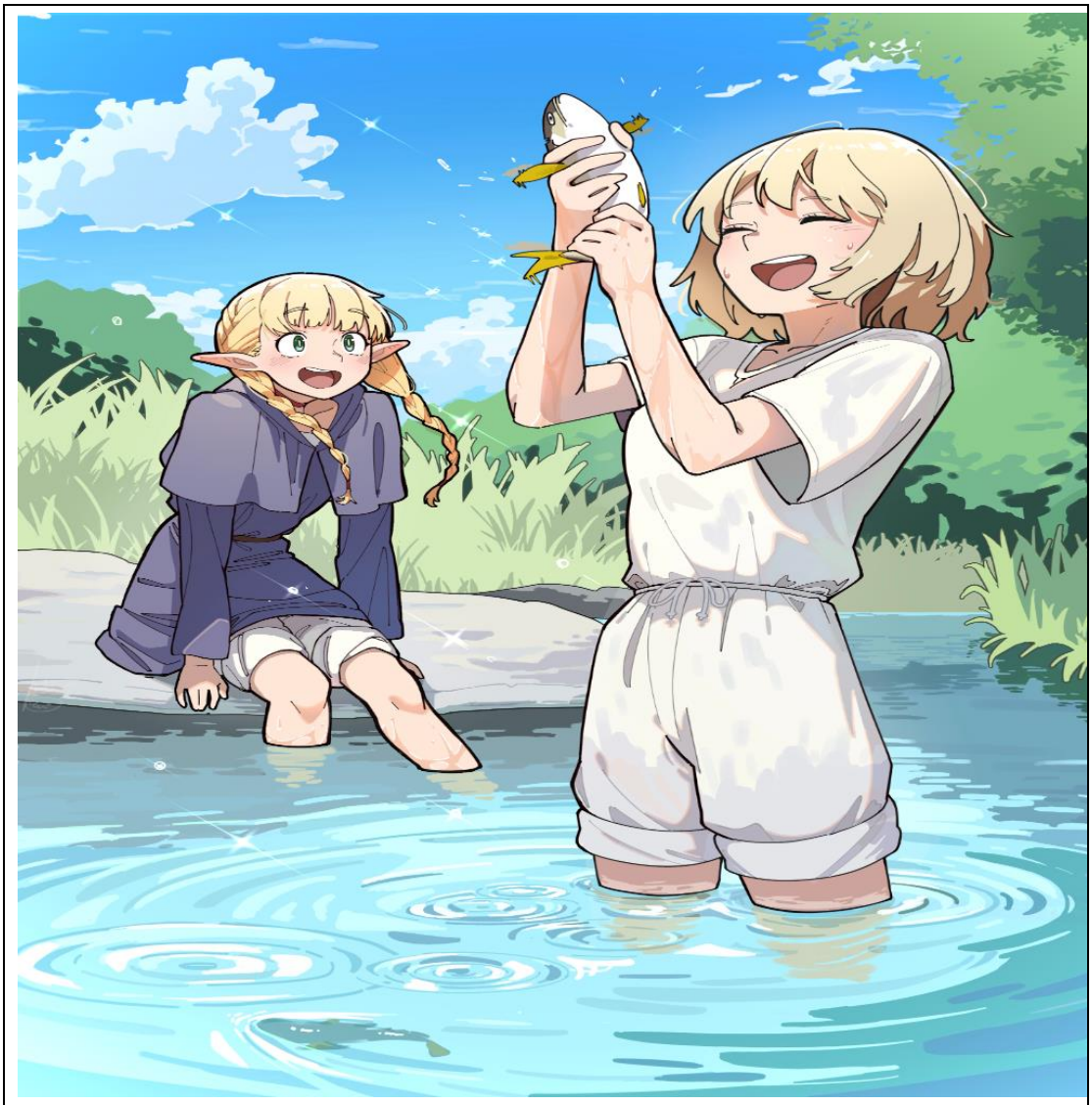


Image1



Image2



Image3



Image4



Image5



Image6



Image7



Image8

Image9

Image10

从 Image0 到 Image10 分别是 1024×1024 像素到 1×1 像素图片

用到牛的模型中，结果如下



实验中存在的问题及解决：

问题 1：生成 Mipmap 序列时，怎样降低代码量、快速生成？

答 1： 因为生成图像与上次生成图像关系是 1/2 * 1/2 像素的关系，所以可以递归生成，降低代码量。

问题 2： 怎样找到屏幕空间像素，去求 du,dv？

答 2：因为屏幕空间像素坐标的取值是连续的，所以使用两个全局变量 last_x,last_y 来直接记录上一次取值屏幕空间像素坐标对应的纹理空间像素坐标，与本次的纹理空间像素坐标做差，得到的 dx, dy 可以认为是 du, dv 经过转化，转化到纹理空间像素的变化量值，然后去求 Mipmap 的层数 L。